

s12 Task System 学习笔记

DAG 任务依赖、持久化函数、状态流转与 FileNotFoundError 调试

核心结论

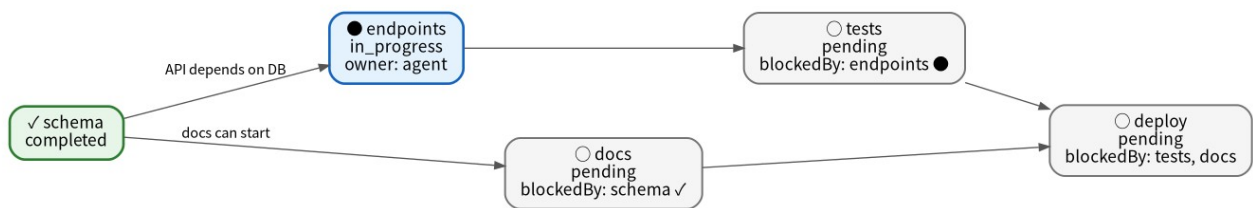
本次对话围绕一个本地 JSON 持久化的 Task DAG 系统展开。重点不是单个语法点，而是理解：Task 对象如何保存/读取、blockedBy 如何表达依赖、claim/complete 如何驱动状态流转，以及为什么依赖文件不存在时必须先 exists() 再 load_task()。

0. 本次对话主线

- 先解释 Task DAG 图：任务节点、箭头依赖、状态颜色、blockedBy、can_start 与 claim。
- 再逐段解释任务系统代码：_save_task、list_tasks、依赖过滤、complete_task、run_list_tasks。
- 随后检查上传代码，定位 Task System 中的运行时错误和设计隐患。
- 最后根据 Traceback 分析 FileNotFoundError：blockedBy 中包含 TASK_ID_PLACEHOLDER，且判断顺序错误。

1. Task DAG：任务依赖模型

DAG = Directed Acyclic Graph，即有向无环图。在任务系统里，节点是任务，箭头表示“后一个任务依赖前一个任务”。例如 schema -> endpoints 表示 endpoints 必须等 schema 完成后才能开始。



概念	含义	代码中的体现
节点	一个 Task	Task(id, subject, status, owner, blockedBy)
边/箭头	依赖方向：A -> B 表示 B 依赖 A	blockedBy 记录前置任务 ID
状态	pending / in_progress / completed	claim_task 与 complete_task 修改 status
可开始	所有依赖都 completed	can_start_task(task_id)
领取	把 pending 任务分配给 owner 并进入 in_progress	claim_task(task_id, owner)

重点

blockedBy 不是“谁被我阻塞”，而是“我被哪些前置任务阻塞”。只要 blockedBy 里还有不存在或未 completed 的任务，当前任务就不能 claim。

2. 总体思维导图：Task System 的组成



3. 持久化函数：Task 对象如何写入 JSON

```
def _save_task(task: Task):
    _task_path(task.id).write_text(
        json.dumps(asdict(task), ensure_ascii=False, indent=2)
    )
```

1. task.id：取出任务 ID，用于生成文件名。
2. _task_path(task.id)：返回 .tasks/<task_id>.json 这种路径。
3. asdict(task)：把 dataclass 对象转换成 dict。
4. json.dumps(..., ensure_ascii=False, indent=2)：把 dict 转成可读 JSON，中文不转义，缩进 2 格。
5. write_text(...)：把 JSON 文本覆盖写入文件。注意它返回“写入字符数”，不是 Path。

推荐写法

把路径、内容、写入分成三步，更适合调试；并显式写 encoding="utf-8"。

```
def _save_task(task: Task) -> None:
    path = _task_path(task.id)
    content = json.dumps(asdict(task), ensure_ascii=False, indent=2)
    path.write_text(content, encoding="utf-8")
```

4. 反序列化函数：list_tasks 如何从 JSON 还原 Task

```
def list_tasks() -> list[Task]:
    return [Task(**json.loads(f.read_text()))
            for f in sorted(TASKS_DIR.glob("*.json"))]
```

- TASKS_DIR.glob("*.json")：扫描任务目录下所有 JSON 文件。
- sorted(...)：保证返回顺序稳定，方便展示和调试。
- f.read_text()：读取 JSON 文本。
- json.loads(...)：把 JSON 字符串解析成 Python dict。
- Task(**data)：字典解包，把 {"id": ..., "status": ...} 展开成 Task(id=..., status=...)。

隐患

Task(**data) 要求 JSON 字段和 Task 构造参数严格匹配：少字段会报缺少参数，多字段会报 unexpected keyword argument。

5. 依赖检查：为什么必须先判断 _task_path(d).exists()

```
deps = [
    d for d in task.blockedBy
    if not _task_path(d).exists() or load_task(d).status != "completed"
]
```

最重要的短路规则

Python 的 or 从左到右执行。当前半部分已经为 True 时，后半部分不会执行。因此要先判断文件是否存在，再决定是否 load_task(d)。

依赖文件状态	代码判断	结果
文件不存在	not _task_path(d).exists() 为 True	不会执行 load_task(d)，安全地判定为 blocked
文件存在但未完成	前半 False，继续 load_task(d).status != completed	判定为 blocked
文件存在且 completed	前半 False，后半 False	依赖满足，不加入 deps

错误写法

`load_task(d).status != "completed" or not _task_path(d).exists()` 会先读文件；如果文件不存在，就在 `exists()` 前已经 `FileNotFoundError`。

6. `complete_task`：完成任务并计算解锁项

```
def complete_task(task_id: str) -> str:
    task = load_task(task_id)
    if task.status != "in_progress":
        return f"Error: Task {task_id} is {task.status}, cannot complete."

    task.status = "completed"
    _save_task(task)

    unblocked = [
        t.subject for t in list_tasks()
        if t.status == "pending" and t.blockedBy and can_start_task(t.id)
    ]
    ...
```

- 状态保护：只有 `in_progress` 的任务才能 `complete`，避免 `pending` 直接跳到 `completed`。
- 先保存再检查 `unblocked`：因为 `can_start_task` 通常会重新读 JSON 文件，如果不先保存，其他任务仍会看到旧状态。
- `unblocked` 的含义更准确地说是“当前所有依赖已满足的 `pending` 任务”，不一定是这次刚刚解锁的任务。
- 你代码中的一个小 bug：拼好了 `msg`，但最后 `return` 了新的字符串，导致 `Unblocked tasks` 信息没有返回。

修复点

`complete_task` 末尾应该 `return msg`，而不是 `return f"Completed {task.id} ({task.subject})"`。

7. `run_list_tasks`：展示层函数

```
def run_list_tasks() -> str:
    tasks = list_tasks()
    if not tasks:
        return "No tasks. Use create_task to add some."
    lines = []
    for t in tasks:
        icon = {"pending": "○", "in_progress": "●",
                "completed": "✓"}.get(t.status, "?")
        deps = f" (blockedBy: {' '.join(t.blockedBy)})" if t.blockedBy else ""
        owner = f" [{t.owner}]" if t.owner else ""
        lines.append(f" {icon} {t.id}: {t.subject} "
                    f"[{t.status}]{owner}{deps}")
    return "
```

```
".join(lines)
```

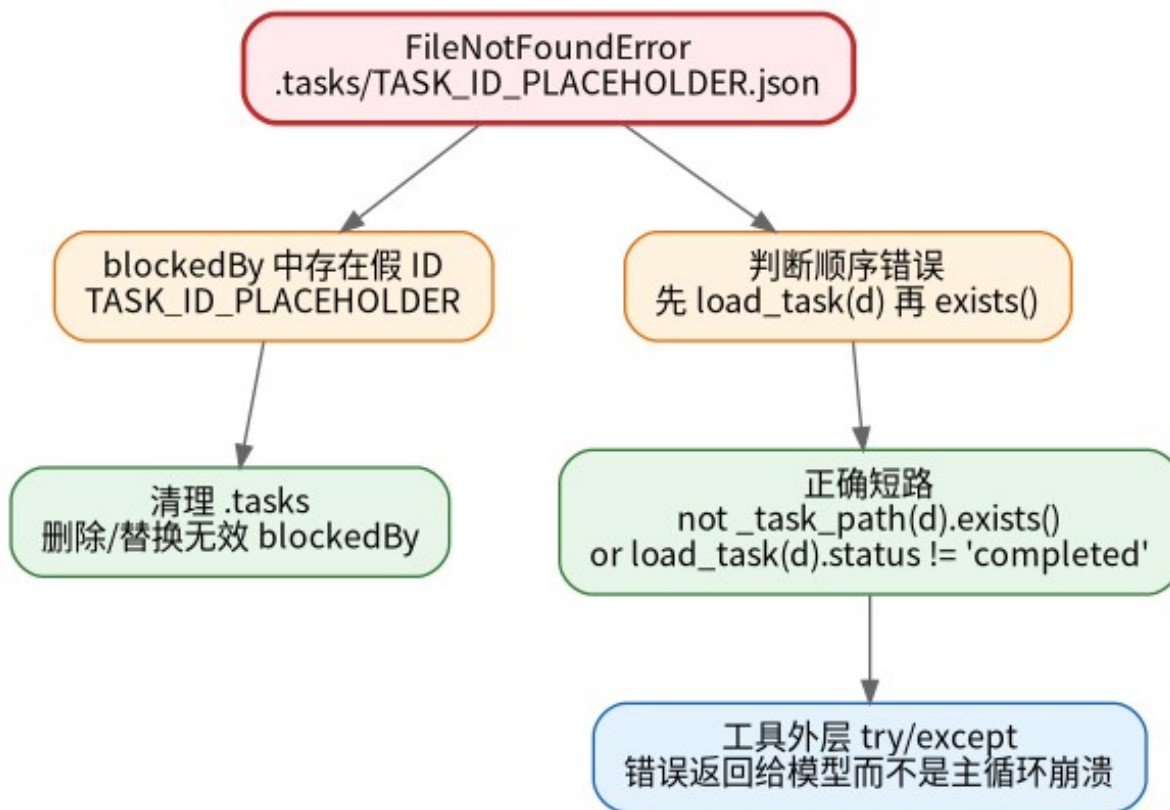
- 这个函数不修改任务，只负责把 Task 对象列表格式化成字符串。
- icon 字典把状态映射成可读符号：○ pending，● in_progress，✓ completed。
- deps 和 owner 都是条件表达式：有内容才显示，没有就显示空字符串。
- "

".join(lines) 把每个任务行合并成多行文本，适合作为工具结果返回给 LLM。

8. 上传代码检查：关键问题汇总

位置	问题	后果	建议修复
_task_path	返回类型注解写成 importlib.resources.path	语义错误，容易误导	删除 from importlib.resources import path, 改成 -> Path
get_task	json.dumps/asdict 括号错： indent 传给了 asdict	调用 get_task 会 TypeError	json.dumps(asdict(load_ta sk(task_id))), ensure_ascii=False, indent=2)
can_start_task	对 load_task(dep_id) 的返回 值调用 exists()	Task 对象没有 exists()	用 _task_path(dep_id).exists()
claim_task	先 load_task(d).status, 再 判断 exists()	依赖文件不存在时 FileNotFoundError	先 not _task_path(d).exists(), 再 load_task(d).status
complete_task	拼好的 msg 没有 return	Unblocked 信息返回丢失	最后 return msg
agent_loop	tool_handler(**block.input) 外层无 try/except	工具内部异常会炸掉整个 agent_loop	捕获异常并作为 tool_result 返回

9. Traceback 复盘：为什么出现 TASK_ID_PLACEHOLDER.json 不存在



- 报错链路：agent_loop -> run_claim_task -> claim_task -> load_task -> Path.read_text。
- 直接原因：某个任务的 blockedBy 中存在 TASK_ID_PLACEHOLDER。程序试图读取 .tasks/TASK_ID_PLACEHOLDER.json。
- 根本原因之一：claim_task 的依赖过滤表达式顺序错误，先 load_task(d)，再 _task_path(d).exists()。
- 根本原因之二：测试数据里残留了 placeholder 依赖 ID，没有替换成真实任务 ID。

处理步骤

1) 修 claim_task / can_start_task 的顺序；2) 打开 .tasks 目录，搜索 TASK_ID_PLACEHOLDER；3) 把无效 blockedBy 改成 [] 或真实任务 ID；4) 给工具执行层加 try/except。

10. 推荐修复补丁

```
def _task_path(task_id: str) -> Path:
    return TASKS_DIR / f"{task_id}.json"
```

```
def load_task(task_id: str) -> Task:
    path = _task_path(task_id)
    return Task(**json.loads(path.read_text(encoding="utf-8")))
```

```

def get_task(task_id: str) -> str:
    return json.dumps(asdict(load_task(task_id)), ensure_ascii=False, indent=2)

def unmet_dependencies(task: Task) -> list[str]:
    return [
        dep_id for dep_id in task.blockedBy
        if not _task_path(dep_id).exists()
        or load_task(dep_id).status != "completed"
    ]

def can_start_task(task_id: str) -> bool:
    task = load_task(task_id)
    return not unmet_dependencies(task)

def claim_task(task_id: str, owner: str) -> str:
    task = load_task(task_id)

    if task.status != "pending":
        return f"Error: Task {task_id} is {task.status}, cannot claim."

    deps = unmet_dependencies(task)
    if deps:
        return f"Blocked by: {deps}"

    task.owner = owner
    task.status = "in_progress"
    _save_task(task)
    return f"Claimed {task.id} ({task.subject})"

def complete_task(task_id: str) -> str:
    task = load_task(task_id)

    if task.status != "in_progress":
        return f"Error: Task {task_id} is {task.status}, cannot complete."

    task.status = "completed"
    _save_task(task)

    msg = f"Completed {task.id} ({task.subject})"
    unblocked = [
        t.subject for t in list_tasks()
        if t.status == "pending" and t.blockedBy and can_start_task(t.id)
    ]
    if unblocked:
        msg += f"
Unblocked tasks: {' ', '.join(unblocked)}"

```

```
return msg
```

工具执行层防崩补丁

主循环中执行工具时加 try/except，把异常变成 tool_result 返回给模型。这样某个工具炸了，agent_loop 不会整体退出。

```
try:
    output = tool_handler(**block.input)
except Exception as e:
    output = f"Error running tool {block.name}: {type(e).__name__}: {e}"
```

11. 复盘检查清单

检查项	是否应该做
_task_path 返回 Path，且不再导入 importlib.resources.path	必须
所有 read_text/write_text 显式 encoding="utf-8"	推荐
所有依赖检查都先 exists() 再 load_task()	必须
.tasks 目录中不再有 TASK_ID_PLACEHOLDER	必须
complete_task 返回 msg，包含 Unblocked 信息	必须
工具调用外层有 try/except	强烈推荐
用 create -> list -> claim -> complete -> list 做一次端到端测试	必须

End - 本笔记根据本次对话与上传代码整理